

Algorithmen: Korrektheit, Effizienz, Statische&Dynamische Finitheit, Terminiertheit, Determiniertheit, Determinismus

Asymptotische (theoretische) Analyse (unendlich große Mengen): $g(n) \sim f(n): \frac{g(n)}{f(n)} \rightarrow 1$

Nur höchste Exponenten entscheidend, Vorfaktoren spielen auch keine große Rolle

Landau-Symbole: $c, c_1, c_2 > 0!$

f ∈ O(g)		f ∈ Θ(g)		f ∈ Ω(g)	
f(n) ≤ c * g(n)		C1 * g(n) ≤ f(n) ≤ c2 * g(n)		f(n) ≥ c * g(n)	
f wächst nicht schneller als g		f wächst genauso schnell wie g		f wächst nicht langsamer als g	
Wenn:	O		Θ		Ω
$\lim_{n \rightarrow \infty} \left(\frac{f(n)}{g(n)}\right) = 0$	f ∈ O(g)		f !∈ Θ(g)		f !∈ Ω(g)
$\lim_{n \rightarrow \infty} \left(\frac{f(n)}{g(n)}\right) = c > 0$	f ∈ O(g)		f ∈ Θ(g)		f ∈ Ω(g)
$\lim_{n \rightarrow \infty} \left(\frac{f(n)}{g(n)}\right) = \infty$	f !∈ O(g)		f !∈ Θ(g)		f ∈ Ω(g)

Reihenfolge: $O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), O(n^k), O(2^n), O(n!)$

Entwurf: Greedy, Rekursion $O(n^2)$, Divide and Conquer $O(n \log n)$, Sweep/Scan $O(n)$, Backtracking $O(m^n)$

Datenstrukturen: Stack (LIFO, push&pop), Queue (FIFO, enqueue, dequeue), Deque, Array, Liste, Bäume, Graphen

Verkettete Listen: head und tail zeigen auf nodes, nodes haben Referenzen auf Vorgänger/Nachfolger, ggf. null

- Einfügen nach dem ersten Element, z.B.: `head.setNext(new Node(data, head.getNext()));`

- Löschen des ersten Elements, z.B.: `head = head.getNext();`

	Speicher	Umsetz.	Stabilität	Beste	mid&worst	Verfahren
BogoSort	In-place	iterativ	instabil	$O(n)$	$O((n+1)!)$	Vertauschen von 2 Zufallselementen
BubbleSort	In-place	iterativ	stabil	$O(n)$	$O(n^2)$	Vertauschen von Elementen, wenn Reihenfolge falsch
SelectionSort	In-place	iterativ	instabil	$O(n^2)$	$O(n^2)$	finde kleinste Element der Restmenge -> vertausche
InsertionSort	In-place	iterativ	stabil	$O(n)$	$O(n^2)$	finde korrekte Position im linken Bereich -> platzieren
MergeSort	Out-of-Place	rekursiv	stabil	$O(n \log n)$	$O(n \log n)$	D&C Prinzip - sortierte Teilmengen zusammenfügen
QuickSort	In-place	rekursiv	instabil	$O(n \log n)$	$O(n \log n)$ worst: $O(n^2)$	D&C Prinzip - Pivot-Element v - 2 Teilmengen größer/kleiner v von links bis $g > v$; von rechts bis $k < v$ -> tausche g und k
Counting Sort	Out-of-Place	iterativ	stabil	$O(n + k)$	$O(n + k)$	Häufigkeit der Schlüssel -> Berechnen der Endposition -> Platzieren

kleine Arrays: InsertionSort; teure Vertauschungen: SelectionSort; große Daten: QuickSort; genug Speicher: MergeSort

Lineare Suche: Iteriere durch die Menge und vergleiche jedes Element; $O(1), O(n), O(n)$

Sentinel-Methode: Füge das gesuchte Element am Anfang oder Ende an und suche am anderen Ende; -1 Vergleich

Binäre Suche: nur Arrays, D&C, Aufsteigende Sortierung vorausgesetzt, vergleiche Mitte dann gehe nach l oder r, $O(1) \mid O(\log n)$

Interpolationssuche: Erweiterung Binärsuche, Teile an Pos P des Schlüssels k von l bis r, $p = l + (r - l) \frac{k - a[l]}{a[r] - a[l]}$, $O(\log(\log n))$

Bäume: Ebene (Abstand Kanten zur Wurzel), Höhe(max. Ebene), Pfad, Pfadlänge, Rang(Anzahl Kindknoten), Ordnung(max. Rang)

Perfekter n-ärer Baum: nichtleere Ebenen haben max. Knotenzahl, Knoten haben genau n Kinder, Blätter haben gleiche Tiefe h,

n^h Blätter, Gesamtzahl Knoten $N = \frac{n^{h+1} - 1}{n - 1}$;

Vollständiger n-ärer Baum: fehlende Knoten rechts unten, Blätter haben Tiefe h oder h-1, Höhe $h = \log_n(N * (n - 1) + 1) - 1$

Suchbäume: Binärbaum, Kinder Knoten i = $2i+1$ und $2i+2$, insert/search/delete: $O(1), O(\log n), O(n) \mid$ Knoten/Höhe/trav. $O(n)$

Traversierung: Inorder(L-K-R), Preorder(K-L-R), Postorder(L-R-K), Level-Order(Ober->Unten, links->rechts)

n-ärer Suchbaum: max. n Kinder pro Knoten, max. n-1 Schlüssel pro Knoten, Schlüssel aufsteigend sortiert

Balancierte Bäume: mittlerer Fall: $O(\log n)$, schlechter $O(n)$

AVL-Baum: Differenz der Höhen max. 1, Höhe h hat mind. $Fib(h+1)$ Blätter und $Fib(h+1)-1$ Knoten, Speicherkomplexität: $O(n)$

Balance oberer Knoten	Balance unterer Knoten	Einfügen/ Löschen	Suchen	$O(1)$	$O(\log n)$
-2	-1	Rechts	Rotieren	$O(1)$	$O(1)$
+2	+1	Links	Balancieren	$O(1)$	$O(1)$
+2	-1	Rechts-links	Einfügen	$O(1)$	$O(\log n)$
-2	+1	Links-rechts	Löschen	$O(1)$	$O(\log n)$

B-Bäume: Tiefe und Suche: $\sim \log_2 n$, mehr Informationen pro Knoten -> bessere Ladezeit der Knoten,

k-äre Bäume: m. Ordnung, $k=2m+1$, Tiefe: $\sim \log_k n$, Schlüssel zw. m und $2m$, Kinder: zw. $m+1$ und $2m+1$,

Anzahl Knoten eines B-Baums, mit Ordnung m und Höhe h: $N_{min} = 1 + \frac{1}{2}((m+1)^h - 1)$ $N_{max} = \frac{1}{2m}((2m+1)^{h+1} - 1)$

B+-Bäume: Nutzdaten nur in den Blättern, innere Knoten nur Schlüssel, oft Blätter mit Referenzen auf Nach/Vorgänger

Heap: MaxHeap oder MinHeap: Elternknoten immer größer/kleiner gleich den Schlüsseln, sollte balanciert sein, Array mögl.

Kinder mit i = $2i+1$ und $2i+2$, Parent mit $\text{floor}((i+1)/2)$, Versickern und Aufschwimmen für Heap Bedingung

Heapify: Array: Elemente rechts von $\text{floor}((n-2)/2)$ sind Blätter, Aufbau von unten nach oben, starte Array im floor i—nach links

HeapSort: Inplace, iterativ, instabil, $O(n \log n)$; Aufbau des Heaps: $O(n)$, Versickern: $O(\log n) \rightarrow O(n) + n * O(\log n) = O(n \log n)$

Priority Queue: mit Heap lösbar, mit swim Prio sicherstellen, entnommen wir Element mit höchster Prio, Elemente haben Prio

Treap: Elemente mit Schlüssel + Prio, Schlüssel erfüllen Suchbaum-Bedingung, Prios erfüllen Heap-Bedingung, Wurzel hat höchste Prio, Linker Unterbaum enthält kleinere Schlüssel, Rechter Unterbaum größere Schlüssel, exakt 1 Treap möglich,

Einfügen: Suche Einfügestelle für Schlüssel, füge Blatt hinzu, falls Heap Bedingung verletzt -> Rotieren

Suchen/Löschen: wie Binärbaum

Löschen der Wurzel: Ersetze Wurzel durch größtes(kleinstes) Element im linken(rechten) Unterbaum -> ggf. Rotieren

Hashtabellengröße: Füllgrad $\alpha = \frac{n}{m} = \frac{\text{Anzahl zu speichernder Elemente}}{\text{Größe der Tabelle}}$ es gilt: $\alpha < 1$ generell: $\alpha < 0,8$, besser $\alpha < 0,5$

Hashfunktion: gleiche Schlüssel zum gleichen Hashwert, jeder Index muss berechenbar sein (Surjektivität), effizient berechenbar

Modulare Hashfunktion: Indes des Schlüssels s in Tabellengröße m: $i = h(s) = s \% m$ (darauf achten, dass i nicht negativ ist)

Anforderungen m: keine Zweierpotenz, keine Zehnerpotenz, sollte prim sein, gute Zahlen: 5657, 55931, 563359

Gleitkomma-Schlüssel: Möglichkeit 1 Normieren: $i = h(s) = \text{floor}\left(\frac{s-l}{r-l} * m\right)$ l=min. Wert Schlüsselbereich, r = max. Wert

Möglichkeit 2: Modulares Hashing auf Binärdarstellung der Kommazahl. Interpretation von Mantisse und Exponent als Ganzzahl.

String-Schlüssel: Interpretiere als große Integerzahl, modulares Hashing für Integer, Horner Schema geeignet für große Zahlen

Hashkollision: unterschiedliche Schlüssel führen evtl. zum selben Index, je größer das Array umso geringer die Wahrscheinl.

- **Verkettung:** in Arrayindex verkettete Liste angelegen für gleiche Hashwerte, Liste finden $O(1)$, in Listenlänge k $O(k)$

- **Offenes Hashing:** Platz belegt -> freier Platz gesucht, Ende muss definiert sein (z.B. null), beim Löschen Einträge verschieben

- **Lineares Sondieren:** freier Platz mit Schrittweite k gesucht: $i_1 = (h(s) + k) \% m \mid i_2 = (h(s) + 2k) \% m \mid \dots$;

k darf kein Teiler von m sein, m muss prim sein, schlechterer Fall $O(n)$, 1 Arrayplatz muss frei bleiben, Clusterbildung schlecht

- **Quadratisches Sondieren:** $h_1(s)$ für Index, $h_2(s)$ für Inkrement

Ø Anzahl Sondierungen	Element wird gefunden	Element nicht gefunden
Lineares Sondieren	$\frac{1}{2} \left(1 + \frac{1}{1-\alpha}\right)$	$\frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2}\right)$
Quadratisches Sondieren	$\frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right)$	$\left(\frac{1}{1-\alpha}\right)$

Adjazenzmatrix: für n Knoten: Matrix mit n Zeilen und n Spalten; Eintrag(i,j) zeigt ob Kante von i nach j existiert; Symmetrische

Matrix bei ungerichtetem Graph; gewichteter Graph enthält Eintrag das Gewicht der Kante; Speicherk. $O(n^2)$ Zugriff $O(1)$

Adjazenzliste: Array/Liste der Knoten; Einträge halten Liste mit Knoten der Kantenverbindung, bei ungerichteten Graphen: beide

Richtungen prüfen oder symmetrisch speichern; gewichteten: kann Kantengewichtung der Verbindung mitgespeichert werden

Breitensuche: Start: beliebiger Knoten -> verarbeite Nachbarn/Nachfolger -> danach deren Nachb./Nachf. | effizient mit Queue

Tiefensuche: Start: beliebiger Knoten -> verarbeite Nachbar/Nachfolger -> Tiefensuche des NB/NF | effizient mit Stack

Dijkstras Algorithmus: setze alle Knoten Abstand: ∞ -> Abstand Start: 0 -> gehe NB: setze kleinster Abstand, speicher Vorgänger

-> weiter am geringsten Abstand: Besuche NB -> wiederhole mit seinem NB usw. -> Kürzester Weg: rückwärtsgehen